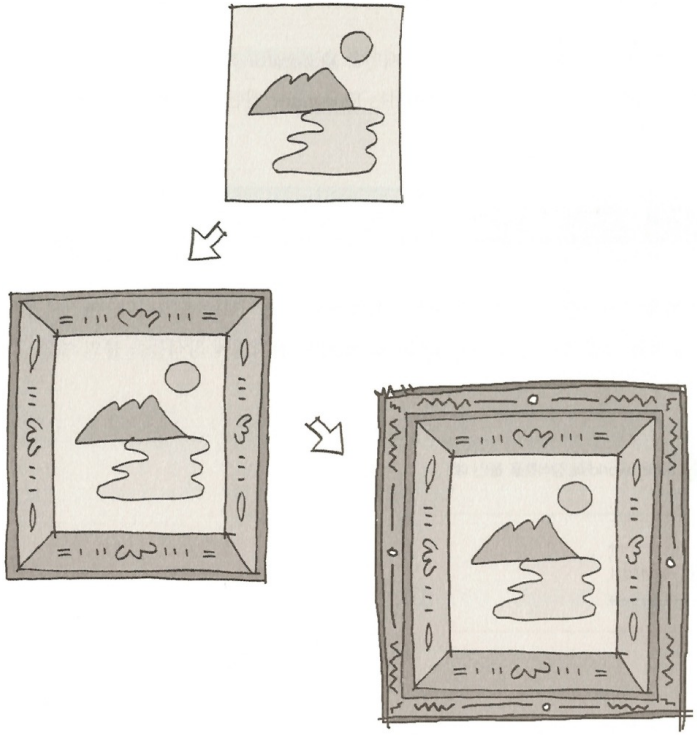
디자인 패턴

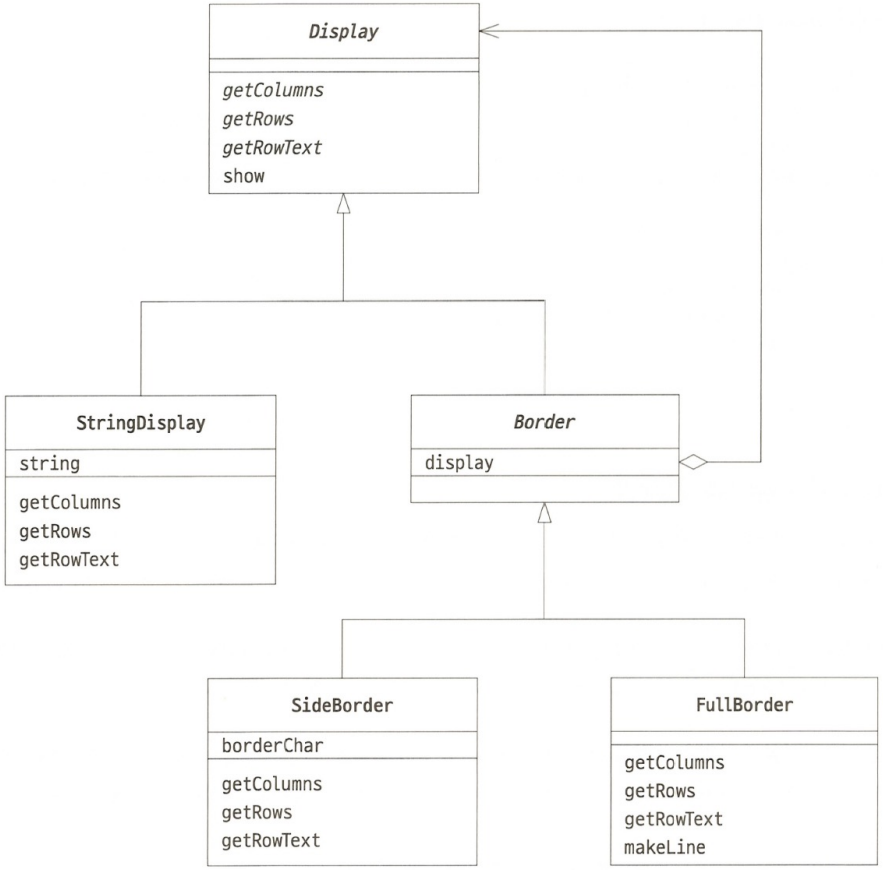
2026. 4. 7 w4
미림 마이스터고

남궁성
seong.namkung@gmail.com

8. Decorator – 장식과 내용물을 동일시



8. Decorator – 장식과 내용물을 동일시



8. Decorator – 장식과 내용물을 동일시

같은 클래스를 **상속 & 포함**. 왜?

```
public class FilterInputStream extends InputStream {
    protected volatile InputStream in;
    protected FilterInputStream(InputStream in) {
        this.in = in;
    }

    public int read() throws IOException {
        return in.read();
    }
    ...
}
```

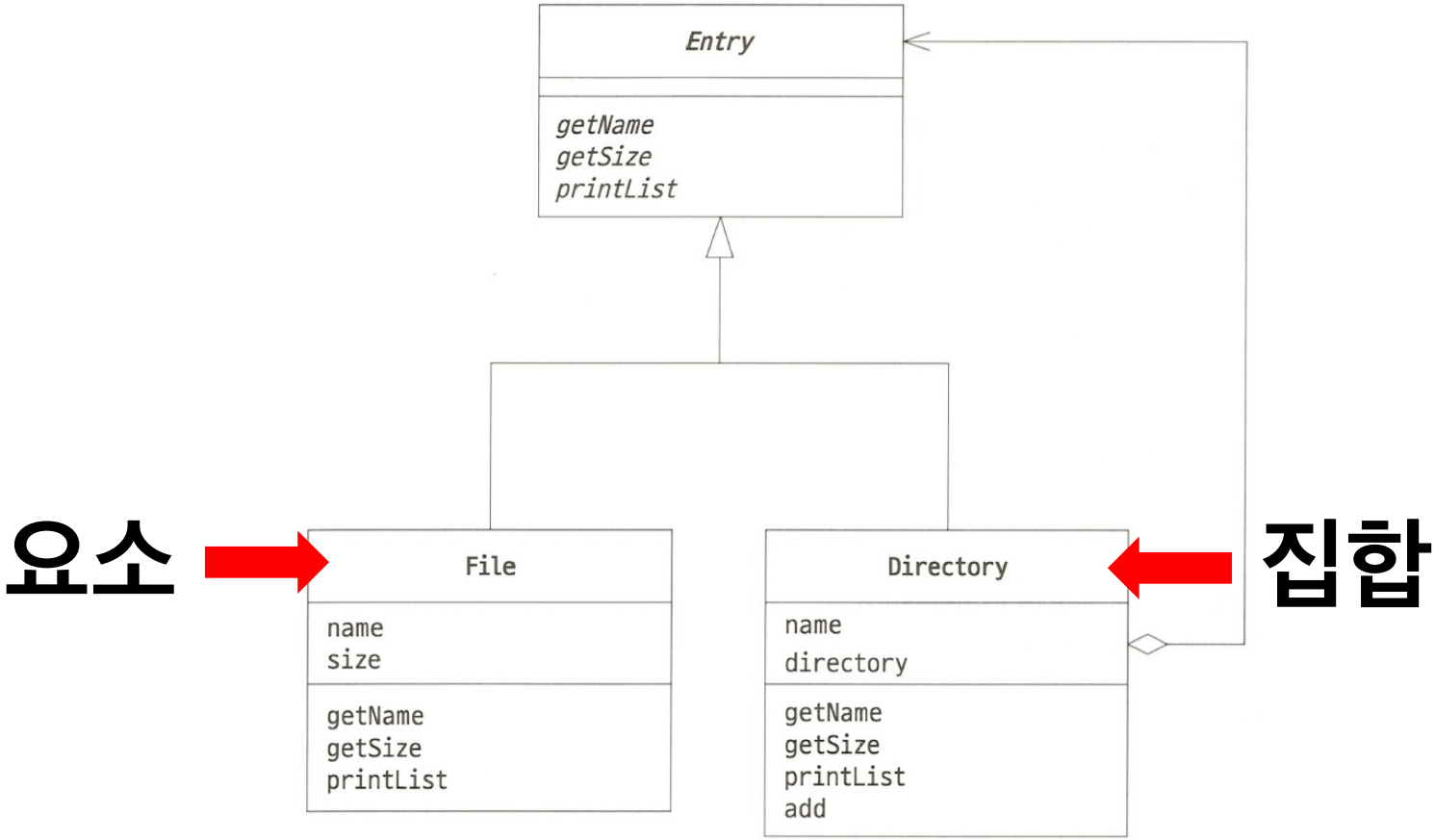
8. Decorator – 장식과 내용물을 동일시

데코레이터가 주입된 객체의 역할을 해야하기 때문(다형성)
(Ring에 Watch를 넣은 것도 Watch이어야 함)

```
InputStream in  = new BufferedInputStream(System.in); // 장식에 알맹이(System.in)이 끼우기(주입)
InputStream in2 = new DataInputStream(new BufferedInputStream(System.in)); // 장식을 여러개 가능
InputStream in3 = new BufferedInputStream(new DataInputStream(System.in)); // 장식의 순서 바꾸기 가능
```

```
Watch w = new Ring(new Watch()); // Ring이 Watch를 상속해서 가능

// 같은 클래스(Watch)를 상속 & 포함
class Ring extends Watch {  // 상속  no usages
    Watch w; // 포함  1 usage
    Ring(Watch w) {  no usages
        this.w = w;
    }
}
class Watch{}  no usages
```

9. Composite – 파일과 폴더를 동일시



9. Composite – 파일과 폴더를 동일시

```
// List<File>에는 파일만 저장 가능
List<File> files = new ArrayList<>();
files.add(new File( name: "readme.md",  size: 100)); // OK
files.add(new Directory( name: "tmp")); // ERROR
```

```
// List<Entry>에 디렉토리와 파일 모두 추가 가능
List<Entry> directory = new ArrayList<>();
directory.add(new Directory( name: "tmp"));
directory.add(new File( name: "readme.md",  size: 100));
```

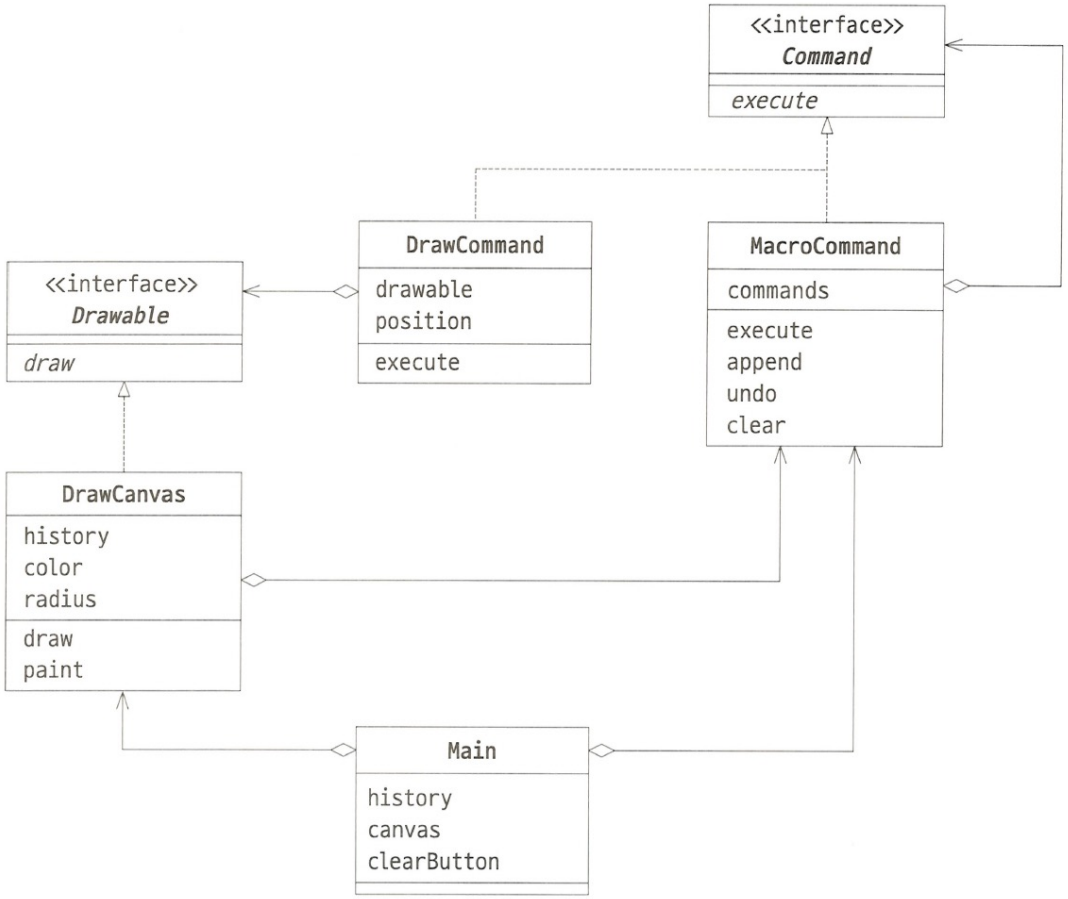
9. Composite – 파일과 폴더를 동일시

```java
public class File extends Entry {  no usages
    private String name;  2 usages
    private int    size;  2 usages

    public File(String name, int size) {  no usag
        this.name = name;
        this.size = size;
    }

    public String getName() { return name; }  1
    public int getSize() { return size; }  1 usag
    protected void printList(String prefix) {
        System.out.println(prefix+"/"+ this );
    }
}
```

```java
public abstract class Entry {  no usages
    public abstract String getName();  1 usage
    public abstract int getSize();  1 usage
    protected abstract void printList(String prefix);
    public void printList() {  no usages
        printList( prefix: "");
    }
    public String toString() {
        return getName() + " (" + getSize() + ")";
    }
}
```

```java
public class Directory extends Entry {  no usages
    private String name;  3 usages
    private List<Entry> directory = new ArrayList<>();

    public Directory(String name) { this.name = name; }
    public String getName() { return name; }  1 usage
    public int getSize() {  2 usages
        int size = 0;
        for (Entry entry : directory)
            size += entry.getSize();
        return size;
    }
    protected void printList(String prefix) {  2 usages
        System.out.println(prefix + "/" + this);
        for (Entry entry : directory)
            entry.printList( prefix: prefix + "/" + name);
    }
    public Entry add (Entry entry){  no usages
        directory.add(entry);
        return this;
    }
}
```

10. Command – 여러 명령을 동일시



10. Command – 여러 명령을 동일시

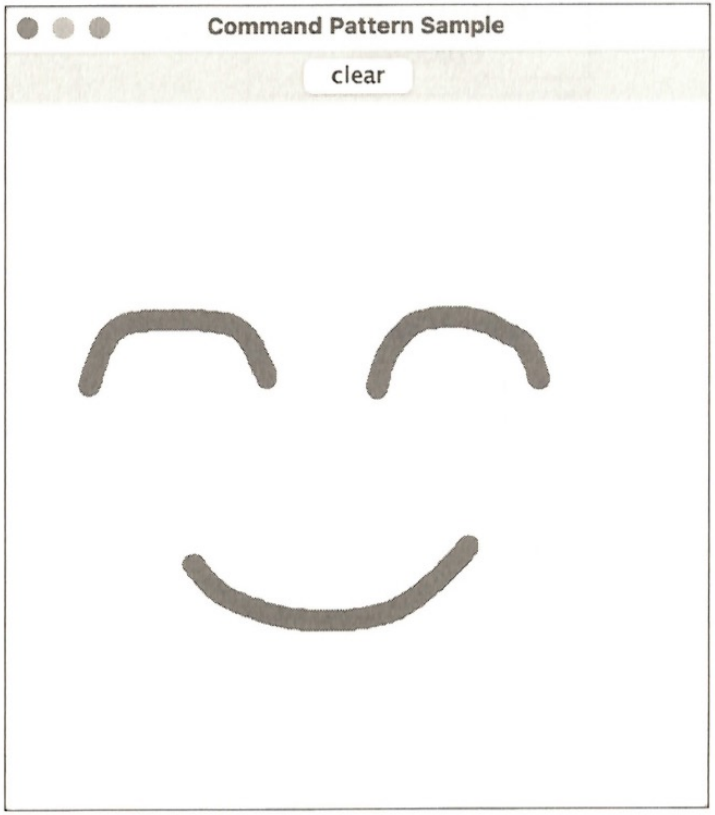
```java
interface Command {  no usages
    public abstract void execute();  no usages
}
```

```java
class DrawCommand implements Command {  no usages
    protected Drawable drawable;  2 usages
    private Point position;  3 usages

    public DrawCommand(Drawable drawable, Point position) {
        this.drawable = drawable;
        this.position = position;
    }

    public void execute() {  1 usage
        drawable.draw(position.x, position.y);
    }
}
```

```java
interface Drawable {  2 usages
    public abstract void draw(int x, int y);
}
```

```java
class MacroCommand implements Command {  no usages
    private Deque<Command> commands = new ArrayDeque<>();  5 usage

    public void execute() {  1 usage
        for (Command cmd : commands)
            cmd.execute();
    }

    public void append(Command cmd) {  no usages
        if (cmd == this)
            throw new IllegalArgumentException("infinite loop");
        commands.push(cmd);
    }

    public void undo() {  no usages
        if (!commands.isEmpty())
            commands.pop();
    }

    public void clear() { commands.clear(); }  no usages
}
```

10. Command – 여러 명령을 동일시




```java
class MacroCommand implements Command {  no usages
    private Deque<Command> commands = new ArrayDeque<>();  5 usage

    public void execute() {  1 usage
        for (Command cmd : commands)
            cmd.execute();
    }

    public void append(Command cmd) {  no usages
        if (cmd == this)
            throw new IllegalArgumentException("infinite loop");
        commands.push(cmd);
    }

    public void undo() {  no usages
        if (!commands.isEmpty())
            commands.pop();
    }

    public void clear() { commands.clear(); }  no usages
}
```